

AbortController in JavaScript

Overview

AbortController is a Web API that allows you to abort one or more asynchronous operations (like fetch requests) when needed. It provides a way to cancel ongoing operations programmatically.

Basic Concept

- Creates a signal that can be passed to async operations
- When aborted, the signal notifies all operations listening to it
- Useful for canceling requests, stopping timers, or cleaning up resources

Core Components

1. AbortController Instance

```
const controller = new AbortController();
```

2. Signal Property

- `controller.signal` - A read-only signal object
- Pass this signal to operations you want to control
- Properties:
 - `signal.aborted` - Boolean indicating if already aborted
 - `signal.reason` - The reason provided when aborted

3. Abort Method

- `controller.abort()` - Cancels all operations using this controller's signal
- Can optionally pass a reason: `controller.abort(reason)`
- Triggers `abort` event on the signal

Common Use Cases

1. Canceling Fetch Requests

```
const controller = new AbortController();

fetch("https://api.example.com/data", {
  signal: controller.signal,
})
  .then((response) => response.json())
  .then((data) => console.log(data))
  .catch((err) => {
    if (err.name === "AbortError") {
      console.log("Fetch aborted");
    } else {
      console.error("Fetch error:", err);
    }
  });

// Cancel the request
controller.abort();
```

2. Timeout for Requests

```
const controller = new AbortController();
const timeoutId = setTimeout(() => controller.abort(), 5000);

fetch(url, { signal: controller.signal })
  .then((response) => {
    clearTimeout(timeoutId);
    return response.json();
  })
  .catch((err) => {
    if (err.name === "AbortError") {
      console.log("Request timed out after 5 seconds");
    }
  });
```

3. Search Input Debouncing

Cancel previous search requests when user types new input:

```

let controller;

searchInput.addEventListener("input", (e) => {
  // Cancel previous request
  if (controller) {
    controller.abort();
  }

  // Create new controller for new request
  controller = new AbortController();

  fetch(`/search?q=${e.target.value}`, {
    signal: controller.signal,
  })
    .then((response) => response.json())
    .then((data) => displayResults(data))
    .catch((err) => {
      if (err.name !== "AbortError") {
        console.error("Search error:", err);
      }
    });
});

```

4. Aborting Multiple Operations

One controller can abort multiple operations:

```

const controller = new AbortController();
const signal = controller.signal;

Promise.all([
  fetch("/api/users", { signal }),
  fetch("/api/posts", { signal }),
  fetch("/api/comments", { signal })
])
  .then((responses) => Promise.all(responses.map((r) => r.json())))
  .catch((err) => {
    if (err.name === "AbortError") {
      console.log("All requests aborted");
    }
  });

// Abort all three requests
controller.abort();

```

Signal Events

Listening to Abort Events

```
const controller = new AbortController();

controller.signal.addEventListener("abort", () => {
  console.log("Operation was aborted");
  console.log("Reason:", controller.signal.reason);
});

controller.abort("User cancelled the operation");
```

Checking Abort Status

```
if (controller.signal.aborted) {
  console.log("Already aborted");
} else {
  // Proceed with operation
}
```

React Integration Example

Cleanup in useEffect

```
useEffect(() => {
  const controller = new AbortController();

  fetch("/api/data", { signal: controller.signal })
    .then((response) => response.json())
    .then((data) => setData(data))
    .catch((err) => {
      if (err.name !== "AbortError") {
        setError(err);
      }
    });

  // Cleanup function - abort on unmount
  return () => controller.abort();
}, []);
```

Browser Support

- Modern browsers (Chrome 66+, Firefox 57+, Safari 12.1+, Edge 16+)
- Node.js 15+ (with experimental flag), 16+ (stable)

Key Points to Remember

1. **One-time use:** Once aborted, a controller cannot be reused. Create a new one for new operations.
2. **Error handling:** Always check for `AbortError` to distinguish aborts from actual errors.
3. **Cleanup:** Remember to abort controllers in cleanup functions (e.g., React useEffect cleanup).
4. **Signal sharing:** The same signal can be passed to multiple operations.
5. **No side effects:** Aborting a completed operation has no effect.

Common Patterns

Pattern 1: Request with Timeout

```
function fetchWithTimeout(url, timeout = 5000) {
  const controller = new AbortController();

  const timeoutId = setTimeout(() => {
    controller.abort(new Error("Request timeout"));
  }, timeout);

  return fetch(url, { signal: controller.signal }).finally(() =>
    clearTimeout(timeoutId)
  );
}
```

Pattern 2: Cancellable Promise

```
function cancellablePromise(promise) {
  const controller = new AbortController();

  return {
    promise: promise.then((result) => {
      if (controller.signal.aborted) {
        throw new Error("Operation cancelled");
      }
      return result;
    }),
    cancel: () => controller.abort(),
  };
}
```

Alternatives Before AbortController

Before AbortController, developers used:

- Manual flags/variables
- Promise wrappers
- Library-specific cancellation (like Axios cancel tokens)

AbortController is now the standard way to handle cancellation in JavaScript.